Modular Morse Certificates for Compressed Unknot Hierarchies

Polynomial-time collapse verification, an exponential affine-ranking barrier, and source-authenticated prism witnesses

Research continuation prepared for the ProveIt project
Developed with ChatGPT

12 October 2026

Abstract

Compressed cutting describes a potentially enormous cell complex without listing its cells. A separate problem is to certify that a terminal piece is a 3-ball. We develop an exact, binary-size verifier for supplied discrete Morse matchings whose directed incidence graphs admit synchronized modular rank functions. Modular interval inequalities reduce to two residue classes, whose presence is determined by Euclidean floor sums. Once the modulus, scale, multipliers and phases are fixed, rank offsets are found by a polynomial-time difference-constraint algorithm. Matching validation, critical-cell counts and proof replay do not expand any represented interval.

We prove that restricting ranks to contiguous piecewise-affine functions in fixed source coordinates can require exponentially many pieces, even for a collapsible path with constantly many interval recipes. Modular ranks avoid this obstruction. A product construction lifts the result to regular three-dimensional prism complexes and also supplies relative collapse certificates onto a small base. The topological conclusion uses authenticated regular PL geometry; a raw CW packet, a boundary matrix, or a successful arithmetic check alone is insufficient.

The standard-library implementation passes 18 tests with 90,121 counted cases, including 480 independent three-dimensional source comparisons and 444 relative-collapse checks. Binary stress tests reach 16,385-bit namespace bounds. Paired supplied-matching queries retain both large represented-size gains and small-input regressions. Native source regularization, pattern transport, matching discovery and global hierarchy bounds remain separate obligations. This work establishes a polynomial encoded certificate operator, not a general quasi-polynomial unknot recognizer.

Reference repository revision: `704600626606a9097cfd89d4e3d2bbc63c8dd1cb`.
Targeted source review; no production patch applied and no native knot benchmark run.

Contents

1 Research position and main contributions **3**
1.1 The specific missing operation . . . 3
1.2 What the contribution is, and is not . . . 3
1.3 Relation to Lackenby's complexity target . . . 4
1.4 Audit boundary . . . 4

2 The encoded object and the geometry contract **4**
2.1 Cell families and covers . . . 4
2.2 Matching orientation . . . 5
2.3 Three different sizes . . . 5

3 From a rank certificate to a collapse **6**

4 Affine ranks and exact offset synthesis **6**

5 Exact residue counting in binary intervals **7**

6 Synchronized modular Morse ranks **8**
6.1 Coordinate covariance . . . 10
6.2 Composition across family components . . . 10

7 Modular paths and an exponential affine barrier **11**
7.1 A constant-recipe source . . . 11
7.2 A binary rank for the whole path . . . 11
7.3 The barrier is in the certificate language . . . 12

8 Relative collapse and genuine three-dimensional witnesses **13**
8.1 Product cells rather than an abstract chain complex . . . 13
8.2 Adding a base Morse matching . . . 13
8.3 The affine obstruction survives in dimension three . . . 14
8.4 Authenticating the finite base . . . 14

9 Implementation, verification and measurements **15**
9.1 Delivered algorithms . . . 15
9.2 Completed test evidence . . . 15
9.3 Paired query measurements and their scope . . . 16
9.4 Huge binary namespaces . . . 16
9.5 Reproduction . . . 17

10 Safe integration into the hierarchy programme **17**
10.1 Three checks, not one . . . 17
10.2 The raw-CW regularity trap . . . 18
10.3 Collapse does not transport a boundary pattern automatically . . . 18
10.4 Failure semantics . . . 18
10.5 Suggested staging . . . 18

11 What would make the full cost quasi-polynomial? **19**

12 Further research questions **19**

13 Conclusion **21**

A Algorithmic pseudocode and certificate format **21**

B Claim and evidence ledger **22**

1 Research position and main contributions

1.1 The specific missing operation

The maintained ProveIt raw-cut constructor retains binary cell families, actual attaching occurrences and reconstruction charts. Its more recent source-parallelity construction already includes product handles in all relevant dimensions, an inherited core with at most $30T$ handles, and a linear core-port bound for a T-tetrahedron source. Thus a new report should not present either raw cutting or a bounded inherited core as a newly missing primitive [6, 7].

The distinction between a bounded inherited core and an admissible patterned hierarchy is material. The parallelity article supplies an explicit empty-cut solid-torus example whose inherited core is not admissible. The raw-cut article separately warns that homology is not a ball certificate. We address the latter distinction: how can one certify a terminal piece as a ball without expanding its potentially huge cell structure?

A certificate of elementary collapse is more informative than an equality of homology groups. The main algorithmic challenge is that even writing one line per elementary collapse would destroy binary compression. We instead prove that a succinct ranking of the directed matching graph certifies the existence of a collapse sequence. The sequence itself is never output in expanded form.

1.2 What the contribution is, and is not

The central results are the following.

1. **Exact modular rank compilation.** For supplied interval matchings and fixed synchronized modular parameters, all rank-offset constraints can be computed in polynomial encoded time and solved exactly. Independent replay uses residue counts rather than the producer's offset solver; see Theorem 6.2.

2. **A representation barrier.** A constant-recipe path forces at least $\lceil N/2 \rceil$ contiguous affine rank pieces in its original vertex indexing, although its encoded size is $O(\log N)$; see Theorem 7.3. This is a lower bound on a particular certificate language, not a hardness claim about path recognition.

3. **A genuine three-dimensional lift.** Products of modularly indexed paths with small triangulated bases admit succinct relative Morse certificates. With an appropriate disk base they yield compact PL 3-balls with exponentially many represented cells; see Theorems 8.1 and 8.2.

4. **A precise topological interface.** Arithmetic success gives acyclicity. An authenticated regular PL cellulation and a single critical vertex then give a ball certificate. The geometry, the pattern and the original source must remain independently authenticated; see Corollary 3.2 and Section 10.

Discrete Morse theory, elementary collapses, modular inverses, Euclidean floor sums and difference constraints are classical ingredients. The claimed contribution is their proved combination for this compressed interface, the fixed-coordinate exponential barrier, and the associated implementation and source tests. Novelty is relative to the material specifically inspected here. This report does not establish a comprehensive historical priority claim.

The controlled prism family is deliberately topologically transparent. A direct product argument already recognizes its members as balls. Its role is to establish a genuine three-dimensional compressed source, an exact representation-size separation, and reproducible verifier measurements. It is not evidence of improved performance on difficult knot diagrams.

1.3 Relation to Lackenby's complexity target

Oxford's 2021 announcement states the $n^{c \log n}$ objective [1]. Lackenby's July 2026 preprint distinguishes operation costs from iteration counts: Section 9 gives $L(g+1)^L$ in terms of maximal hierarchy length and pattern complexity, without bounding both quantities there. Its refined Section 10 bounds hierarchy length using initial q-complexity; this is not itself a logarithmic bound in diagram size [2]. Our operator addresses one local certificate cost. Neither it nor these controlled examples provide the remaining global estimates.

1.4 Audit boundary

The reference revision is recorded above. We inspected the maintained raw-cut, source-parallelity and boundary-shelling articles, project and report indexes, the incoming directory listing and intake instructions, and retrieved portions of the earlier moment-coded first-return report [6, 7, 8, 9, 10]. The four binary archives visible in the incoming directory were not extracted in this session. No full repository checkout, production test rerun or native source adapter is claimed. `SOURCE_AUDIT.md` distinguishes pinned reads from earlier reads of the moving branch.

2 The encoded object and the geometry contract

2.1 Cell families and covers

Definition 2.1 (Interval cell packet)**.** An interval cell packet has q disjoint cell families

$$\mathcal{F}_\alpha = \{F_\alpha(i) : \ell_\alpha \le i \le h_\alpha\}, \qquad \ell_\alpha, h_\alpha \in \mathbb{Z},$$

with a fixed dimension $d_\alpha \in \{0,1,2,3\}$ for each family. It has r cover recipes

$$U(i) \succ V(\epsilon i + b), \qquad \ell \le i \le h, \qquad \epsilon \in \{-1,1\}, \tag{1}$$

where $d_U = d_V + 1$. Every integer in a recipe's domain names an actual codimension-one incidence. The packet includes m matching subintervals of these recipes.

All interval endpoints and offsets are binary integers. Families denote disjoint cells, not aliases of cell occurrences. A valid cover inventory is a set: two recipes may not describe the same actual ordered cell pair. A matching uses each cell at most once, whether that cell is the upper or lower member of its pair.

The arithmetic packet does not, by itself, establish that its cover relation is the complete face relation of a geometric complex. That is an external source obligation. For the topological results we require a finite regular PL cell complex, with compatible cell charts for elementary free-facet collapses. Simplicial complexes and the polytopal products used below satisfy this requirement. When the conclusion is a 3-ball, the source must additionally authenticate a compact PL 3-manifold.

This contract excludes the common but invalid substitution of a cellular boundary matrix for actual incidence. A matrix coefficient can vanish because two occurrences cancel; that does not make either occurrence disappear geometrically. It also excludes a cell whose closure identifies different points of its own boundary. The raw ProveIt cut representation deliberately retains such source identifications [6]; regularity cannot simply be asserted for it.

2.2 Matching orientation

Orient every unmatched cover from upper cell to lower cell. Reverse a matched cover, so its arrow points from the matched lower cell to the matched upper cell. Write $G_{\mathcal{M}}$ for the resulting directed graph on all represented cells. A cell used by no matching pair is *critical*.

A *rank certificate* is an integer-valued function R on the cells such that

$$R(x) \geq R(y) + 1 \quad \text{for every arrow } x \longrightarrow y. \tag{2}$$

Ranks need not be nonnegative. The graph is finite, so strict decrease alone excludes directed cycles. A common additive shift can make all ranks nonnegative when desired.

Every reversed matched recipe is still affine. The inverse of $j = \epsilon i + b$ is $i = \epsilon j - \epsilon b$, and its domain is the interval image of the original matching subinterval. This is why both slope signs are included from the start.

Proposition 2.2 (Encoded preparation)**.** *Valid matching data can be checked, all directed recipes produced, and all critical-cell counts computed without iterating over a represented interval. There are at most $r + 2m$ directed recipes and at most $q + 2m$ critical intervals. A conservative implementation uses $O(r^2 + (q + m)\log(q + m + 1) + r + m)$ integer comparison/arithmetic operations, apart from dictionary or indexed-name bookkeeping.*

Proof. Check dimensions and recipe endpoints. For duplicate covers with the same family pair, intersect their domains. Equal slopes either agree everywhere on the overlap or nowhere. Opposite slopes agree only at a possible integer solution of

$$(\epsilon_1 - \epsilon_2)i = b_2 - b_1.$$

Thus pairwise overlap checking is sufficient, including a singleton collision.

For each matching interval record its upper domain and lower image. Sort all occupied intervals within each family and reject overlaps. Their complements give critical intervals, and an interval $[u, v]$ contributes $v - u + 1$ critical cells in its family's dimension. There are $2m$ occupied records in total.

Within a cover, deleting k disjoint matching intervals leaves at most $k + 1$ unmatched intervals. Summing over covers gives at most $r + m$ unmatched recipes and m reversed recipes. All work uses endpoints, additions, signs and comparisons, not physical cell counts. □

The implementation's name maps use Python dictionaries. The polynomial comparison-model statement can instead use sorted names with indexed arrays or balanced search trees. No worst-case constant-time hashing assumption is required for the theorem. Cell dimension is fixed at most three in the delivered verifier.

2.3 Three different sizes

It is useful to keep the following quantities separate:

$$S_0 = q + r + m, \qquad B = \max \text{bits(source integer)}, \qquad N_{\text{cell}} = \sum_{\alpha} (h_\alpha - \ell_\alpha + 1).$$

Here $\text{bits}(z) = 1 + \lceil \log_2(|z| + 1) \rceil$ includes a sign allowance. Polynomial encoded time means polynomial in S_0 and B, together with certificate-parameter bit lengths. It does not mean polynomial in $\log N_{\text{cell}}$ alone when the number of recipes grows. Explicit oracles in the package have hard expansion caps and are not used in the compressed path.

3 From a rank certificate to a collapse

The following special case of discrete Morse theory is included with a direct proof to make the exact topological conclusion visible. The broader theory is classical [3, 4, 5].

Lemma 3.1 (Acyclic matching relative to a critical subcomplex)**.** *Let K be a finite regular PL cell complex with the free-facet collapse property in Section 2. Let L be a subcomplex consisting exactly of the critical cells of a matching $\mathcal{M}$. If $G_{\mathcal{M}}$ is acyclic, then K collapses onto L using precisely the pairs of $\mathcal{M}$.*

Proof. If $K = L$, there is nothing to do. Otherwise choose the largest dimension d of a cell outside L. Every such d-cell σ is matched to a $(d-1)$-face τ_σ: it cannot be critical, and it cannot be matched upwards. Indeed an upper cell outside L would contradict maximality of d, while an upper cell in the subcomplex L would put σ in L.

Consider the directed graph on these d-cells with an arrow $\beta \to \sigma$ whenever $\beta \neq \sigma$ is another d-coface of τ_σ. Each such arrow expands to a directed path

$$\beta \longrightarrow \tau_\sigma \longrightarrow \sigma$$

in $G_{\mathcal{M}}$, because the first incidence is unmatched and the second matched. This graph is acyclic and has a source σ. Therefore τ_σ has no other d-coface.

It has no higher coface outside L by the choice of d, and no coface in L because it lies outside L. Thus τ_σ is a free face of the maximal cell σ. Delete the pair by an elementary collapse. The remaining cells form a subcomplex, the remaining matching remains acyclic, and the critical subcomplex is still L. Induction completes the collapse. A nonempty zero-dimensional remainder outside L would have to be matched but could not be paired in either direction, so this cannot obstruct termination. □

Corollary 3.2 (Authenticated ball certificate)**.** *Suppose an interval packet is authenticated as a regular PL cellulation of a compact PL 3-manifold M. If its matching has exactly one critical vertex and no other critical cells, and it has a rank certificate satisfying (2), then M is a PL 3-ball.*

Proof. The single vertex is a subcomplex. Strict rank decrease implies acyclicity, so Lemma 3.1 collapses M to that vertex. The PL collapsible-manifold theorem of Whitehead gives the ball conclusion. For polytopal cells, compatible subdivisions interpret the elementary collapses in the PL category. The source must supply the hypotheses needed for this interpretation; it is not inferred from the interval arithmetic [4, 5]. □

A single critical vertex without acyclicity is insufficient. Section 7 gives a particularly small false-positive trap: a path component together with directed cycle components has exactly one critical vertex, but is not collapsible. The delivered tests explicitly reject it.

The ball conclusion is also not an unknot conclusion by itself. A hierarchy may cut a knotted exterior into terminal balls. The meaningful assertion includes the surfaces used, the inherited patterns and the rule that propagates the terminal result back to the source. Those data are not supplied by a Morse rank.

4 Affine ranks and exact offset synthesis

We first isolate the simpler certificate language, both as a baseline and to state exactly what the later obstruction defeats.

Definition 4.1 (Family-affine rank)**.** For each family F, choose integers a_F, h_F and set

$$R(F(i)) = a_F i + h_F.$$

A contiguous piecewise-affine rank is obtained by first partitioning each source family into consecutive integer intervals and giving each part its own formula.

For a directed recipe $U(i) \to V(\epsilon i + b)$ on $[\ell, h]$, condition (2) is equivalent to

$$h_U - h_V \geq C_e, \qquad C_e = \max_{i \in \{\ell, h\}} \big(1 + a_V(\epsilon i + b) - a_U i\big). \tag{3}$$

The endpoint maximum is exact because the expression is affine in i.

Lemma 4.2 (Difference-constraint solver)**.** *Given q variables and constraints $h_U \geq h_V + C_e$ with integral C_e, a feasible integral solution exists if and only if every directed cycle in the constraint graph has total weight at most zero. Feasibility and a solution are found in $O(qe)$ integer operations for e constraints. If $C = \max_e |C_e|$, a solution can be chosen in $[0, (q-1)C]$.*

Proof. A positive cycle is impossible by summing its inequalities. Conversely, add a source with a zero-weight edge to every variable and orient the constraint edge from V to U with weight C_e. When no positive cycle exists, deleting cycles does not decrease a walk's weight. A longest walk can therefore be taken simple after the added source, with at most $q-1$ ordinary edges. Its maximum weight, including the zero walk, gives h_U and the stated bound.

Starting all labels at zero, synchronously relax every constraint. After k rounds, a label is the maximum weight of a walk with at most k edges, also allowing a zero start at every vertex. If the labels stabilize, they solve all constraints. If there is no positive cycle, stabilization occurs by the qth round. Otherwise a feasible fixed point cannot exist. This is the longest-path version of the usual difference-constraint algorithm. □

Proposition 4.3 (Complete fixed-slope affine compiler)**.** *For supplied slopes a_F, a family-affine rank exists exactly when the system (3) is feasible. The compiler either supplies offsets or correctly reports that this fixed-slope template fails. Verification requires only the two endpoint inequalities per directed recipe.*

The distinction in the last sentence matters. A failed fixed-slope template is not evidence of a directed cycle, a noncollapsible complex or a nonball. It is merely failure of that chosen certificate form. Even allowing arbitrary slopes on every contiguous piece does not prevent the exponential phenomenon proved in Section 7.3.

5 Exact residue counting in binary intervals

Define the Euclidean floor sum

$$F(n, M, a, b) = \sum_{j=0}^{n-1} \left\lfloor \frac{aj + b}{M} \right\rfloor, \qquad n \geq 0, \quad M \geq 1, \tag{4}$$

where a, b may have either sign. We include the proof and executable recurrence; no new floor-sum algorithm is claimed.

Lemma 5.1 (Euclidean floor sums)**.** *The value in (4) is computable using $O(1 + \log(M + 1))$ Euclidean-division steps, with integer bit lengths polynomially bounded in those of n, M, a, b.*

Proof. Write $a = q_a M + a_0$ and $b = q_b M + b_0$ with $0 \le a_0, b_0 < M$. Then

$$F(n, M, a, b) = q_a \frac{n(n-1)}{2} + q_b n + F(n, M, a_0, b_0).$$

This identity also handles negative input coefficients because Euclidean division uses nonnegative remainders.

Assume now $0 \le a, b < M$. If $an + b < M$, every summand is zero. Otherwise $a > 0$. Put $an + b = Mn' + b'$ with $0 \le b' < M$. Counting integer lattice points below the line $y = (ax + b)/M$, now by horizontal rather than vertical slices, gives

$$F(n, M, a, b) = F(n', a, M, b'). \tag{5}$$

For a direct check, a point in row $k \in \{1, \dots, n'\}$ has $x \ge \lceil (Mk - b)/a \rceil$. The number of allowed $x \in \{0, \dots, n-1\}$ is

$$n - \left\lceil \frac{Mk - b}{a} \right\rceil = \left\lfloor \frac{M(n' - k) + b'}{a} \right\rfloor.$$

The possible last row has count zero if it is only reached at $x = n$. Summing yields (5).

After swapping (M, a), remove quotient terms again. The positive moduli follow the Euclidean algorithm. The accumulated answer has at most a constant multiple of the combined input bit lengths, since there are n summands with magnitude bounded by $O((|a|n + |b|)/M + 1)$. □

For $[x]_M$ denoting the residue in $\{0, \dots, M-1\}$, the following formula is the key interval primitive.

Proposition 5.2 (Exact threshold occupancy)**.** *Let $\ell \le h$, $0 \le t \le M$, $n = h - \ell + 1$, and $B = a\ell + c$. Then*

$$\#\{i \in [\ell, h] \cap \mathbb{Z} : [ai + c]_M \ge t\} = F(n, M, a, B + M - t) - F(n, M, a, B). \tag{6}$$

This includes $t = 0$, $t = M$, signed a, c and negative interval coordinates.

Proof. For any integer x, the difference

$$\left\lfloor \frac{x + M - t}{M} \right\rfloor - \left\lfloor \frac{x}{M} \right\rfloor$$

is one exactly when $[x]_M \ge t$, and zero otherwise. Substitute $x = a(\ell + j) + c$ and sum. □

Neither a histogram of all residues nor a refinement at every modular wrap is needed. In particular, an interval may contain enormously many wraps. Their presence in a specified class is decided by two floor sums.

6 Synchronized modular Morse ranks

Definition 6.1 (Synchronized modular template)**.** Fix integers $M, D \ge 1$. Give each family integers a_F, c_F and an unknown offset h_F. Define

$$R(F(i)) = D\,[a_F i + c_F]_M + h_F. \tag{7}$$

For every directed recipe $e : U(i) \to V(\epsilon i + b)$ require

$$a_V \epsilon \equiv a_U \pmod{M}. \tag{8}$$

The parameters M, D, a_F, c_F constitute the template. The offsets are to be synthesized or verified.

The synchronization condition is sufficient structure, not a property possessed by every acyclic interval graph. It may also be stricter than necessary on a singleton interval. The verifier rejects a nonsynchronized template rather than making an unsupported inference. Signed coordinate changes are supported; no orientation of a topological fibre is encoded by the arithmetic slope.

For an arrow recipe set

$$\delta_e = [a_V b + c_V - c_U]_M\,, \qquad r(i) = [a_U i + c_U]_M\,, \qquad n_e = h - \ell + 1.$$

Synchronization gives

$$[a_V(\epsilon i + b) + c_V]_M = [r(i) + \delta_e]_M\,.$$

Consequently the directed rank difference is exactly

$$R(U(i)) - R(V(\epsilon i + b)) = h_U - h_V - D\delta_e + DM\,\mathbf{1}_{\{r(i) \geq M - \delta_e\}}. \tag{9}$$

Let w_e be the number of indices in the indicated wrap class, computed by (6). Formula (9) has only two possible values, however large the domain.

Theorem 6.2 (Exact modular compilation and independent replay)**.** *For a prepared interval matching and a synchronized modular template, define*

$$C_e = 1 + D\delta_e - DM\,\mathbf{1}_{\{w_e = n_e\}}. \tag{10}$$

A rank of form (7) *exists if and only if*

$$h_U \geq h_V + C_e \tag{11}$$

is feasible. The offsets can be computed, or nonexistence for this template established, in

$$O\big(e(1 + \log(M+1)) + qe\big) \tag{12}$$

integer arithmetic operations for $e \leq r + 2m$ directed recipes. A supplied proof can be independently replayed in $O(e(1 + \log(M+1)))$ such operations. Both costs are polynomial in the complete binary input size, with no factor N_{cell}.

Proof. If even one nonwrapping occurrence exists, it requires $h_U - h_V \geq 1 + D\delta_e$. That inequality also suffices for wrapping occurrences, whose rank difference is larger by DM. If all occurrences wrap, the precise requirement is instead $h_U - h_V \geq 1 + D\delta_e - DM$. This proves the exact equivalence with (10).

Use Proposition 5.2 to determine whether $w_e = n_e$, then apply Lemma 4.2. For independent verification, do not invoke the offset solver: compute w_e again and check the nonwrapping inequality when $w_e < n_e$, and the wrapping inequality when $w_e > 0$. These are exactly all rank differences actually present. In particular a class absent from the recipe imposes no spurious inequality.

Floor-sum work gives the first term of (12); relaxation gives the second. Since $0 \leq \delta_e < M$, we have $|C_e| \leq 1 + DM$. A feasible offset vector can be chosen with $O(\text{bits}(D) + \text{bits}(M) + \log(q+1))$ bits per entry. Coefficient-index products and floor sums have polynomial bit length by Lemma 5.1. Include preparation from Proposition 2.2 to obtain the full encoded bound. □

For clarity, the verifier and compiler share the tested exact residue-count primitive. They do not share offset discovery. Independence here means independent inequality replay with the producer and solver disabled, not two completely disjoint arithmetic libraries or a formal proof of the Python implementation.

Example 6.3 (A wrap cannot be replaced by endpoint sampling)**.** The residue function $[ai + c]_M$ is generally not affine on an interval. It may cross zero repeatedly even when both endpoint values look harmless. The verifier therefore never applies an endpoint-only test to a modular expression. It uses endpoint tests only for affine ranks, and threshold occupancy for modular ranks. Both empty wrap classes and intervals containing both classes occur in the exhaustive arithmetic checks.

6.1 Coordinate covariance

Proposition 6.4 (Reflection transport)**.** *Reindex a family by $i = \rho j + s$, with $\rho \in \{-1, 1\}$ and a bijection between its old and new integer domains. Transport a modular rank by*

$$a'_F = a_F \rho, \qquad c'_F = a_F s + c_F, \qquad h'_F = h_F.$$

After transporting every cover and matching domain, all rank inequalities and critical-cell counts are unchanged. The transformed data remain interval recipes with slopes ± 1.

Proof. Substitution gives literal equality of ranks on the same cells. Affine bijections map domains and matching images to intervals and have affine inverses of slope ± 1. Thus the entire directed matching graph is merely relabelled. □

This avoids treating the numerical order within a source family as a topological invariant. At the same time, Section 7.3 shows why silently changing the family partition is not free when comparing certificate sizes.

6.2 Composition across family components

Proposition 6.5 (Component-local rank certificates)**.** *Form the directed graph on families by forgetting the integer domains of directed recipes. Suppose each strongly connected component has a valid local rank certificate for its internal recipes, using its own affine or modular parameters. These certificates combine into a polynomial-size global rank certificate; coefficient synchronization is not required across distinct components.*

Proof. For each component C choose an integer lower bound l_C and upper bound u_C on all its local ranks. For affine ranks, endpoint minima and maxima suffice. For modular ranks use

$$\min_{F \in C} h_F \le R(F(i)) \le \max_{F \in C} h_F + D_C(M_C - 1).$$

These bounds need not be sharp. Shift local ranks by $-l_C$ so they lie in $[0, H_C]$, where $H_C = u_C - l_C$. The condensation graph is acyclic. In reverse topological order define

$$T_C = \max(\{0\} \cup \{T_E + H_E + 1 : C \to E,\ C \neq E\}).$$

Add T_C to all shifted ranks in C. Internal inequalities are preserved. For an intercomponent arrow from C to E, even the smallest rank in C exceeds the largest in E. Offset magnitudes have bit length bounded by the local bit lengths plus $O(\log(q + 1))$. □

This proposition is an additional proved extension, not an implemented multi-component certificate format in the delivered code. It identifies a useful next compiler improvement: insist on synchronization only where actual family-level recurrence requires it, not across a directed acyclic assembly of independent pieces.

7 Modular paths and an exponential affine barrier

7.1 A constant-recipe source

Let $N \geq 2$, $1 \leq s < N$ and $0 \leq p < N$. Begin with vertices v_i, $0 \leq i < N$, and an edge e_i from v_i to $v_{[i+s]_N}$ for every i except

$$t = [p - s]_N .$$

Match e_i with its tail v_i. There is one critical vertex, v_t. The source has one vertex family, at most two edge intervals, identity tail incidences and at most two translation formulas for successor incidences, split at $N - s$. Thus it has $O(1)$ recipes and $O(\log N)$ source bits.

Proposition 7.1 (Exact topology and acyclicity of the modular source)**.** *If* $\gcd(s, N) = 1$*, the source is a regular cellulation of an interval, and its matching collapses to* v_t*. If* $d = \gcd(s, N) > 1$*, it is a path together with* $d - 1$ *cycle components; the matching graph has directed cycles despite its single critical vertex.*

Proof. The permutation $i \mapsto i + s \pmod N$ consists of d disjoint orbits. Deleting the outgoing edge at t breaks exactly the orbit containing p into a path. The other orbits remain cycles. A matched tail followed by the unmatched head incidence traverses each edge in the successor direction, giving precisely the asserted directed paths and cycles. □

Distinct edges may have the same pair of endpoints for a small cycle. They remain different regular cells; the complete cell-pair incidence inventory distinguishes them. None has a self-identified endpoint because s is nonzero modulo N.

7.2 A binary rank for the whole path

When $\gcd(s, N) = 1$, let $b = s^{-1} \pmod N$. The traversal index of vertex i is

$$j(i) = [b(i - p)]_N .$$

It starts at zero at p and ends at $N - 1$ at t. Put

$$r(i) = [-bi + bp + N - 1]_N = N - 1 - j(i). \tag{13}$$

For every existing edge, $r(i) \geq 1$ and $r([i + s]_N) = r(i) - 1$.

Theorem 7.2 (Succinct path rank)**.** *The formulas*

$$R(v_i) = 2r(i), \qquad R(e_i) = 2r(i) - 1$$

certify the modular path matching. They are synchronized modular ranks with $M = N$*,* $D = 2$*, common multiplier* $-b$*, common phase* $bp + N - 1$*, and offsets* 0 *and* -1*. Their total encoded size is* $O(\log N)$*.*

Proof. The matched arrow $v_i \to e_i$ decreases rank by one. The unmatched arrow $e_i \to v_{[i+s]_N}$ also decreases it by one. The successor recipes have slope one, so synchronization holds. The modular inverse has $O(\log N)$ bits and is found by the extended Euclidean algorithm. □

The verifier does not need a separate hand-coded exception for the last step of the path: the deleted edge at t removes the bad occurrence from the source domain, and the exact wrap-class query recognizes the remaining interval inequalities.

7.3 The barrier is in the certificate language

Take $N = 2m + 1$, $s = 2$ and $p = 0$. The directed path visits the vertices in the order

$$0, 2, 4, \ldots, 2m, 1, 3, 5, \ldots, 2m - 1. \tag{14}$$

Theorem 7.3 (Exponential contiguous-affine ranking barrier)**.** *For the source indexing above, every rank satisfying* (2) *and represented by contiguous piecewise-affine functions on the single vertex family requires at least*

$$\left\lceil \frac{N}{2} \right\rceil$$

affine pieces. The source packet and a modular rank certificate both have $O(\log N)$ bits.

Proof. Along (14), a rank decreases strictly between consecutive vertices because each step is a two-arrow directed path through an edge. In particular every even-indexed vertex has larger rank than every odd-indexed vertex.

In the natural numerical order $0, 1, \ldots, 2m$, every three consecutive vertex ranks therefore form a strict peak or a strict valley. An affine function on three consecutive integers has equal successive differences and cannot have this property. Every contiguous affine piece contains at most two vertices, so at least $\lceil N/2 \rceil$ pieces are needed. The constant-recipe source and Theorem 7.2 give the upper bound for the alternative modular encoding. □

The lower bound allows arbitrary real slopes and intercepts on the affine pieces, hence also arbitrary integer ones. It is not restricted to the slopes tried by the finite test suite.

Remark 7.4 (What the barrier does not say)*.* A parity-based arithmetic-progression refinement, or a source order changed to traversal order, also describes this particular path compactly. The theorem does not prohibit those languages. It prohibits silently assuming that a small number of *contiguous* affine pieces exists in the fixed source coordinate system. It is not a lower bound for every succinct rank format, and it does not prove general succinct acyclicity hard.

For a concrete small example, with $N = 9$ the traversal order is $0, 2, 4, 6, 8, 1, 3, 5, 7$. The vertex ranks from Theorem 7.2, listed in numerical order, are

$$16, 6, 14, 4, 12, 2, 10, 0, 8.$$

Their alternating peaks and valleys already rule out any affine interval containing three vertices. Increasing N changes the number of represented cells without changing the number of source recipes.

8 Relative collapse and genuine three-dimensional witnesses

8.1 Product cells rather than an abstract chain complex

Let P be the modular path of Section 7, with $\gcd(s, N) = 1$. Let A be a finite simplicial complex of dimension at most two, with k nonempty simplices, explicitly supplied. Give $P \times A$ its product cellulation. A product cell is either $v_i \times \sigma$ or $e_i \times \sigma$. Its codimension-one faces come from exactly one factor at a time. These are actual polytopal cells, not just formal tensor factors of a boundary matrix.

Lift every path matching pair over every simplex σ:

$$v_i \times \sigma \longleftrightarrow e_i \times \sigma \qquad (i \neq t).$$

No cell over v_t is used by this lifted matching. The interval compiler splits the vertex namespace at t into at most three intervals. This constant-size split isolates the terminal copy of A without following the path traversal order.

Theorem 8.1 (Relative product-collapse certificate)**.** *The lifted matching collapses $P \times A$ onto $\{v_t\} \times A$. Its rank certificate has $O(k)$ families and recipes and $O(k(\log N + \log(k+1)))$ bits, apart from the explicit base description. One valid rank is*

$$R(v_i \times \sigma) = 2r(i) + 2\dim\sigma + 1,$$
$$R(e_i \times \sigma) = 2r(i) + 2\dim\sigma.$$

Proof. The matched arrow over a fixed σ decreases rank by one. An unmatched head incidence decreases it by $2 - 1 = 1$. A base-facet incidence decreases $\dim\sigma$ by one and hence rank by two. These are all directed cover types. The critical cells are precisely the terminal copy of A, which is a subcomplex. Lemma 3.1 gives the relative collapse.

There are constantly many path families and recipes. Taking their product with the k base cells and its explicit face relations gives the asserted size. In dimension at most three, the number of simplex facets per cell is bounded, so the base incidence size is $O(k)$. □

This relative conclusion does not require A to be a disk or even a manifold. It is useful for a verified product collar or sleeve attached to a retained base. It is a PL collapse, not a homeomorphism replacing a three-dimensional patterned piece by a lower-dimensional base; the difference is crucial for later hierarchy operations.

8.2 Adding a base Morse matching

Suppose A has an acyclic matching $\mathcal{M}_A$. Lift that matching only over v_t, in addition to the path matching. The critical cells of the product matching are exactly v_t times the critical cells of $\mathcal{M}_A$.

Let $H(\sigma)$ be a rank on the finite base's directed matching graph, chosen in $[0, k-1]$ by topological ordering. For a common integer $D \geq k + 1$, put

$$\begin{aligned} R(v_i \times \sigma) &= Dr(i) + 2\dim\sigma + 1 && (i \neq t), \\ R(e_i \times \sigma) &= Dr(i) + 2\dim\sigma, \\ R(v_t \times \sigma) &= H(\sigma). \end{aligned} \tag{15}$$

Theorem 8.2 (Product Morse compiler and ball witnesses)**.** *The rank* (15) *certifies the product matching. If A is a triangulated PL 2-disk and $\mathcal{M}_A$ has one critical vertex and no other critical cells, then $P \times A$ is a compact PL 3-ball with a polynomial encoded Morse certificate. For fixed A, the represented cell count is $(2N-1)k$ while certificate size is $O(\log N)$.*

Proof. Away from the terminal layer, the directed incidences are the three types checked in Theorem 8.1. Their rank decreases are 1, $D-1$, and 2 respectively. On the terminal layer they are exactly the arrows certified by H.

An edge-head incidence entering the terminal layer has $r(i) = 1$, and its upper rank is $D + 2\dim\sigma$. The target rank is at most $k-1$, so this arrow strictly decreases as well. There are no other interface arrows. Thus the matching is acyclic and its critical-cell inventory is the base's critical inventory.

The geometric source is literally $[0,1] \times A$, with its regular product cells. For a disk base it is a PL 3-manifold, and the Morse certificate invokes Corollary 3.2. Alternatively its ball topology follows directly from the product description. Both routes identify the same genuine source; neither authenticates an unrelated knot-exterior packet. □

The delivered producer uses $D = 2k + 4$, a deliberately conservative bound. It discovers the offsets by Theorem 6.2, rather than hard-coding (15). Thus the finite product construction also checks that the general offset compiler finds a certificate whose existence has an independent proof.

8.3 The affine obstruction survives in dimension three

Corollary 8.3 (Three-dimensional fixed-coordinate barrier)**.** *For the step-two odd-N paths and any fixed disk base with a base matching to one vertex, the lifted three-dimensional matching inherits an $\Omega(N)$ lower bound for contiguous affine pieces on its vertex-product families, in the original numerical path coordinates. A modular certificate still has $O(\log N)$ size for the fixed base.*

Proof. Fix a base vertex z. The path-matching arrows through $v_i \times z$ and $e_i \times z$ contain the full directed path (14). Therefore the same alternating-rank argument applies. The constant number of cuts used to isolate t can reduce the lower bound by only a constant number of pieces. The modular upper bound is Theorem 8.2. □

8.4 Authenticating the finite base

The implementation supplies disk fans with a central vertex and an explicitly triangulated boundary polygon. It also independently checks a supplied triangular base: triangles are nondegenerate and not duplicated, each edge has degree one or two, every vertex link is a connected interval or circle, the complex is connected, the boundary is nonempty, and the Euler characteristic is one. The classification of compact surfaces then identifies a disk. This procedure checks source geometry on the finite base rather than trusting a producer's `is_disk` flag.

A triangulated annulus is an important negative test. Its product is not a ball. The deterministic greedy base collapse leaves a three-edge circle, so the delivered example has critical counts $(3,3,0,0)$, not $(1,0,0,0)$. Its rank still verifies, correctly: acyclicity alone does not assert that a matching collapses to a point. With no base matching at all, the product instead collapses relatively to the full annulus, as Theorem 8.1 predicts.

9 Implementation, verification and measurements

9.1 Delivered algorithms

`code/ranked_morse.py` contains the interval schema, preparation, signed floor sums, exact residue thresholds, affine and modular compilers, independent inequality checkers, critical-cell counts, coordinate reflection, JSON serialization and explicitly capped finite oracles. It accepts dimensions zero through three. Arbitrary-size integers above the small-value threshold are serialized in hexadecimal to avoid interpreter-dependent decimal conversion limits.

`code/sources.py` supplies the modular paths, product cellulations, finite base validation and a separate explicit geometric construction. That construction uses cells indexed by a path dimension/rank pair and a base simplex, not by the compressed packet's family names. It compares actual cell dimensions, covers and matching pairs after a checked namespace map.

`code/demo.py` produces and replays a certificate for a source-specific prism ball. The combinatorial rank API returns no generic ball verdict. Only the demo's independently checked product source justifies its source-specific PL-ball statement. There is no adapter accepting an arbitrary `normal-cut-cells-v1` packet and declaring it regular or a ball.

The finite explicit oracles use two different algorithms: topological sorting of the directed Hasse graph, and repeated free-pair deletion using current coface counts. They do not consult the rank certificate. These checks are useful implementation evidence, not substitutes for the proofs of polynomial encoded complexity or the topological source hypotheses.

9.2 Completed test evidence

The retained run used Python 3.13.5 on the Linux environment recorded in `results/validation.json`. All **18 tests passed**, with **90,121 counted cases**, no failures, errors or skips. The cases are inner deterministic or randomized checks, not 90,121 independent theorems or native knot instances.

Check family	Count
Signed floor sums against literal sums	49,725
Residue threshold counts	14,000
Modular recipe constraints against every represented small index	15,000
Affine endpoint constraints	5,000
Difference systems against exhaustive bounded assignments	500
Modular paths, including noncoprime cases	2,708
Closed directed cycle systems	946
Independent three-dimensional source comparisons	480
Relative product-collapse checks	444
Reflected-family certificates	200
Arbitrary small simplicial matchings	1,000
Other source, tampering, replay and binary checks	118

The last row comprises disk-source checks, affine-barrier examples, malformed-packet and rank tampering tests, saved replay, large binary products, a long Euclidean-chain test and an endpoint contract test. The exact category breakdown is machine-readable. A stored 100-bit-parameter prism certificate replays with the producer, offset solver and expansion function disabled. Binary stress tests also disable expansion. A 1,024-bit consecutive-Fibonacci example exercises a long Euclidean chain, rather than only the especially simple step-two residues.

The 480 source comparisons include noncoprime successor steps: they authenticate the represented geometry and check the expected cycle rejection rather than counting every source as a positive ball. The 444 relative tests use no base matching and independently replay the collapse onto the specified terminal base.

9.3 Paired query measurements and their scope

Both timed arms start with the same immutable interval packet and return the supplied matching's acyclicity result together with critical-cell counts. Both include interval validation and orientation. The compressed arm computes a modular inverse, compiles offsets and independently verifies them. The explicit arm expands the directed graph and topologically sorts it. A second identical explicit arm supplies an A/A control.

Source construction and geometric authentication, cleanup, serialization and comparison of answers are outside the timers. Each arm has one warmup and seven measured rounds. Arm order is shuffled within rounds using a fixed seed. All raw samples are in `results/benchmark.json`; ratios are medians of paired explicit/compressed ratios, not ratios selected from best samples.

Source / N	Cells	Compressed ms	Explicit ms	Ratio	A/A
path / 3	5	0.128	0.097	0.78	0.98
path / 65	129	0.137	0.237	1.73	0.97
path / 4,097	8,193	0.153	14.158	93.13	0.85
prism / 3	85	1.687	0.880	0.50	0.90
prism / 9	289	2.084	1.323	0.65	1.07
prism / 33	1,105	2.238	4.093	1.71	1.00
prism / 129	4,369	2.152	14.947	6.81	0.97
prism / 513	17,425	2.206	88.511	40.17	1.01
prism / 2,049	69,649	2.338	646.595	270.38	1.05

For example, the 2,049-vertex prism source represents 69,649 cells. The compressed query median is 2.338 ms and the explicit median is 646.595 ms; the median paired ratio is 270.38, with an A/A control of 1.05. Conversely the three-vertex prism has ratio 0.50: the compressed method is slower. The nine-vertex prism and the smallest path also regress. Timings are environment-dependent, and the naive explicit baseline is not claimed to be the best possible specialized product recognizer.

These results compare two complete *matching queries*, not two complete source-to-knot recognizers. They exclude geometric export from ProveIt, admissibility, decomposing-surface discovery, and hierarchy management. They do not measure Regina or the native orbit engine. In this special family, directly using the source's product theorem would be faster than either Morse query; the benchmark isolates the generic supplied-matching operation.

9.4 Huge binary namespaces

The next measurements use $N = 2^b + 1$, a fixed four-triangle disk fan, and no explicit baseline. Every case retains 85 families, 242 covers and 42 matching recipes. Seven complete compressed-query samples are retained per row. Serialized proof sizes are measured separately from query timing.

Bits in N	Families	Covers	Query ms	Proof bytes
65	85	242	2.617	39,275
1,025	85	242	6.511	239,903
4,097	85	242	41.903	842,783
16,385	85	242	180.006	3,254,303

The final row represents $17(2N - 1)$ cells. Its namespace bound has 16,385 bits; it does not have merely 16,385 cells. The encoded proof grows with integer bit length, and Python's large-integer arithmetic is visibly not constant time. The table supports the claimed absence of interval expansion, not a constant-time or universally fast implementation claim.

9.5 Reproduction

From the package root:

```
python tests/test_research.py
python code/benchmark.py
python code/demo.py --bits 1024
sh build.sh
```

The test runner exits nonzero on failure and writes its validation record. The benchmark regenerates the measured tables. The stored article uses the retained completed run; rerunning timings naturally changes those values. Python 3.10 or later is sufficient for the implementation; the delivered tests were run on the version stated above. The PDF needs a LaTeX installation with the standard packages listed in the source.

10 Safe integration into the hierarchy programme

10.1 Three checks, not one

A future native wrapper must keep the following three assertions distinct:

(i) the packet is the complete regular PL cell structure of the specified source piece;

(ii) the matching is valid and its directed graph is certified acyclic with the claimed critical cells;

(iii) the resulting ball statement, together with an independently authenticated boundary pattern, licenses the particular hierarchy conclusion being requested.

This report implements the general arithmetic part of (ii). It implements (i) only for its controlled explicit-base product source, with independent small-source reconstruction. It does not implement (i) for native normal cuts, nor (iii) for a native hierarchy.

A source wrapper should receive a source identifier, exact piece selection and authenticated geometric data, reconstruct or verify its regularization, and only then bind the matching packet to it. A digest can detect accidental mismatch but cannot replace a proof of that geometric relation. Merely adding a field `regular: true` is not a source certificate.

10.2 The raw-CW regularity trap

The raw-cut article keeps all local source occurrences, even if several identify in the global quotient [6]. Our matching format instead refers to distinct cells of a regular complex. A conversion must therefore address self-identification, coinciding facets, edge and vertex links, and complete cell closures. Removing duplicate matrix entries is not such a conversion.

A promising route is a source-chart or flag-based subdivision with retained provenance. What is needed is more than existence of a finite subdivision: it is a constructive bound on the number of encoded families, proof that their intervals remain manageable, and a complete source-geometric replay. This report does not prove that every native packet has such a polynomial-size regularization. The product sources already have regular charts, so their positive tests cannot establish that missing general statement.

10.3 Collapse does not transport a boundary pattern automatically

A Morse certificate concludes that the *original* authenticated manifold is a ball. It does not turn the final critical point into a representation of the original boundary sphere or its pattern. The source boundary and its pattern must remain available to the terminal pattern checker. The same warning applies to the relative collapse of a product sleeve: homotopy reduction is not a license to identify patterned boundary ports arbitrarily.

The maintained boundary-shelling operation is different: it certifies particular embedded boundary tetrahedron removals preserving the exterior's PL homeomorphism type, with source replay [8]. Our interior Morse certificate can remove cells of several dimensions and is used as a ball witness. It is not an extension of that API that silently preserves all patterned data.

10.4 Failure semantics

Malformed arithmetic data cause certificate rejection. A valid packet with no rank in the attempted template yields Unknown. A directed-cycle witness would disprove acyclicity of that *matching*, not ballness of the underlying source. A known noncollapsible cellulation need not imply its underlying manifold is a nonball. A new matching, a different modular template, a source simplification or a subdivision may succeed.

Only a verified source geometry plus the hypotheses of Corollary 3.2 permits CertifiedBall. None of these outcomes, without a separate knot-theoretic rule, returns Unknot or Knotted. Existing exact recognition fallbacks should remain available after an inconclusive attempt.

10.5 Suggested staging

The package is suitable as a self-contained next report under the project's numbered report collection; its number should be assigned during intake. The code is a reference kernel, not a patch to `fastunknot`. Before production integration, add a native geometric exporter and independent replay, validate it on actual cut pieces and known nonball regressions, account for its full cost, and only then measure end-to-end recognition. The existing production source should not be replaced by the controlled prism source.

11 What would make the full cost quasi-polynomial?

A polynomial verifier is not a polynomial finder of certificates. There are four independent search choices here: regularizing the source, choosing the matching, selecting modular parameters, and choosing hierarchy surfaces. The offset compiler eliminates only one formerly implicit search—the offsets for a fixed template. The following statements make the remaining complexity dependencies explicit.

Proposition 11.1 (Restricted template catalogue)**.** *Suppose a source-authenticated problem of size n comes with a constructible catalogue whose candidates are specified by at most k choices from sets of size at most n^c. Suppose each candidate packet, matching and synchronized template has at most n^c bits in total. Searching the catalogue and checking for an authenticated one-critical-vertex certificate takes*

$$n^{ck+O(1)}$$

bit time. In particular it is quasi-polynomial when $k = O(\log n)$.

Proof. There are at most n^{ck} candidates. By Theorem 6.2 each candidate has polynomial verification and offset-compilation cost in its encoded size. Include the assumed construction and source-authentication cost in the polynomial factor. □

This becomes a restricted ball-recognition theorem only under a coverage theorem: every positive instance in the specified class must have a successful catalogue member. Without coverage, exhaustive catalogue failure is still inconclusive. The product sources have an explicit successful template, but no corresponding coverage theorem is proved for arbitrary normal-cut pieces.

Corollary 11.2 (Conditional hierarchy accounting)**.** *For a complete hierarchy algorithm to which an iteration estimate $L(g+1)^L$ applies, assume that all step costs, including source authentication and necessary searches, are $n^{O(1)}$, that all intermediate encodings have polynomially many bits, that $g \le n^{O(1)}$, and that $L = O(\log n)$. Then its total running time is*

$$\exp(O((\log n)^2)).$$

Proof. Multiply the per-step bound by $L(g+1)^L$ and take logarithms. The iteration exponent contributes $O(L \log(g+1)) = O((\log n)^2)$. □

The corollary is cost accounting, not a claim that its hypotheses hold. This report supplies a polynomial cost for one supplied-matching operator. It does not prove a logarithmic hierarchy bound, polynomial pattern growth, a complete matching catalogue or polynomial work for all other steps. A bound polynomial in a normal vector's *bit length* must also not be confused with one polynomial in the original crossing number unless that intermediate bit length is controlled.

12 Further research questions

1. Source-faithful regularization. Can the source charts of `normal-cut-cells-v1` be converted into a regular PL complex with polynomially many interval families and a polynomial-size independent certificate? The first concrete targets should include self-face pairings and repeated local edges, not just embedded tetrahedra. A negative example for a proposed interval language would be as informative as a positive constructor.

2. Geometrically forced matchings. Which prism and product regions force matching pairs that can be chosen without a global search? Can the exceptional matching decisions be confined to the inherited $O(T)$ core, while all long parallel families use certified formulas? A proof must preserve every attachment to the exceptional core.

3. Small exceptional sets rather than small total cores. An $O(T)$ core is not logarithmically small. Is there a hierarchy measure under which only $O(\log n)$ core locations require branching at a stage, giving an actual coverage theorem for Proposition 11.1? A mere count of all core handles does not yield this conclusion.

4. Discovery of modular parameters. For a strongly connected affine-recipe graph, can useful moduli and multipliers be inferred from cycle translations or source monodromy? The current compiler assumes these parameters. It would be valuable to classify cases where a polynomially generated catalogue is complete.

5. Multiple local moduli. Implement Proposition 6.5, with independent certificates on family strongly connected components. Determine whether this materially reduces synchronization failures on native geometric packets. Cross-component composition should not use a potentially enormous least common multiple when a DAG of local ranks suffices.

6. Beyond a single residue. Can sums or lexicographic tuples of a fixed number of residue functions be checked in polynomial encoded time under useful synchronization hypotheses? The two-class argument no longer applies without additional structure. General residue expressions must not be accepted by endpoint sampling.

7. Relative patterns and collars. Which relative collapse certificates preserve a specified interface subcomplex strongly enough to support a source-authenticated geometric replacement? Theorem 8.1 supplies a collapse onto a base but not a boundary-pattern homeomorphism. The missing transport should be formulated as an explicit theorem, not an implementation flag.

8. Certificate reuse under local hierarchy changes. After a local cut, compression or relabelling, can valid rank proofs on untouched components be reused with only local replay? A dependency graph of source regions and rank inequalities may permit incremental verification, but the source-change authentication must be included in the cost.

9. Native ball and nonball corpus. Build an actual cut-piece corpus containing verified balls, nontrivial homology balls, sphere-boundary nonballs, one-sided-cut cases and complicated attaching maps. Measure exporter cost, certificate discovery, replay and fallback separately. The manufactured product family is insufficient for evaluating deployment.

10. Certificate-language lower bounds. Theorem 7.3 separates contiguous affine and modular ranks. Are there comparably small geometric sources forcing many pieces even after bounded arithmetic-progression refinements, or forcing many synchronized modular components? Such results would guide the choice of a more expressive but still checkable language.

11. Formalization of the arithmetic kernel. Formalize the floor-sum recurrence, exact wrap-class constraints and finite difference-system completeness. Separately formalize the regular-cell collapse lemma. The present Python tests and written proofs are not Lean or Rocq verification.

12. End-to-end hierarchy potentials. Combine certified compressed terminal operators with a potential that controls intermediate source descriptions, pattern growth, reset count and hierarchy depth. The target is a theorem connecting original diagram size to every cost in Corollary 11.2, not merely to the final rank-verification step.

13 Conclusion

The main advance is a rigorous certificate language for an exponentially represented matching graph: modular ranks replace interval refinement by exact arithmetic occupancy tests, and offsets are synthesized without expanding the cells. The exponential affine barrier explains why this change of language is not just a constant-factor optimization. The product lift and independent source oracles demonstrate that the construction concerns actual regular three-dimensional geometry as well as abstract directed graphs.

The contribution has a deliberately sharp boundary. It proves and implements a polynomial encoded acyclicity/collapse-certificate operator; its topological use requires source-authenticated PL geometry. Native regularization, patterned transport and complete hierarchy discovery still need their own theorems and measurements. Keeping those obligations explicit is necessary for this operator to become a trustworthy part of the original quasi-polynomial programme.

A Algorithmic pseudocode and certificate format

The floor-sum recurrence used in the implementation can be written as follows. All divisions are Euclidean, with nonnegative remainder.

```
floor_sum(n, M, a, b):
    answer = 0
    repeat:
        qa, a = divmod(a, M)
        qb, b = divmod(b, M)
        answer += qa*n*(n-1)//2 + qb*n
        y = a*n + b
        if y < M: return answer
        n, b = divmod(y, M)
        M, a = a, M
```

The modular compiler is:

```
compile(packet, M, D, multiplier, phase):
    prepared = validate_and_orient(packet)
    constraints = []
    for each directed recipe U(i) -> V(epsilon*i + shift):
        require a[V]*epsilon == a[U] (mod M)
```

```
        delta = (a[V]*shift + c[V] - c[U]) mod M
        w = exact_count_residue_ge(domain, a[U], c[U], M-delta)
        C = 1 + D*delta
        if w == domain_length: C -= D*M
        constraints.append(h[U] >= h[V] + C)
    h = solve_difference_constraints(constraints)
    if infeasible: return UNKNOWN_FOR_THIS_TEMPLATE
    return (M, D, multiplier, phase, h)
```

The independent rank checker reconstructs directed recipes, tests synchronization, recomputes w, and checks both actually occurring values in (9). It does not call the compiler or use its constraints as an oracle.

The JSON schema `ranked-morse-v1` contains a packet, a rank kind and rank parameters. It is not a native geometric certificate schema. Integers with bit length above 256 use a tagged hexadecimal string. This avoids a hidden dependence on a decimal-digit conversion limit, but it is not a hostile-input service protocol: callers still need appropriate resource limits before accepting arbitrary external JSON.

B Claim and evidence ledger

Claim	Status
Polynomial interval matching preparation	Proved in Proposition 2.2; implemented.
Fixed-slope affine offset compilation	Proved in Proposition 4.3; implemented and tested.
Exact modular threshold occupancy	Proved in Proposition 5.2; exhaustive and randomized arithmetic comparisons.
Complete offset synthesis for a synchronized template	Proved in Theorem 6.2; implemented. Does not discover the matching or template parameters.
Source-authenticated one-critical-vertex ball implication	Corollary 3.2, using classical PL collapse theory. Native source authentication not implemented.
Exponential contiguous-affine barrier	Theorem 7.3; applies to fixed source coordinates, not arbitrary compression languages.
Relative product and three-dimensional lift	Theorems 8.1–8.2; constructed and checked against independent finite geometry.
Local-modulus component composition	Proposition 6.5; proved, not implemented in the delivered format.
Controlled matching-query gains	Completed paired measurements, with tiny-input regressions retained. No recognition speedup claim.
Native regularization, patterned replay and integration	Design obligations only; no adapter or production patch claimed.
General quasi-polynomial unknot recognition	Not established by this report.

References

[1] Mathematical Institute, University of Oxford, *Marc Lackenby announces a new unknot recognition algorithm that runs in quasi-polynomial time*, 2021. https://www.maths.ox.ac.uk/node/38304.

[2] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026, especially Sections 9, 10 and 14. https://arxiv.org/abs/2607.23350.

[3] R. Forman, *A user's guide to discrete Morse theory*, Séminaire Lotharingien de Combinatoire **48** (2002), Article B48c. https://www.mat.univie.ac.at/~slc/wpapers/s48forman.pdf.

[4] J. H. C. Whitehead, *Simplicial spaces, nuclei and m-groups*, Proceedings of the London Mathematical Society, Series 2, **45** (1939), 243–327. The PL collapsible-manifold consequence is also stated in the introduction of [5].

[5] K. A. Adiprasito, B. Benedetti and F. H. Lutz, *Extremal examples of collapsible complexes and random discrete Morse theory*, arXiv:1404.4239; consulted for its account of collapsibility and the PL manifold hypothesis. https://arxiv.org/abs/1404.4239.

[6] ProveIt project, *Full three-dimensional normal cuts in affine cell families*, `Topology/UnknotRecognition/synthesis/cut_cells.tex`. Inspected from the moving main branch on 12 October 2026; see `SOURCE_AUDIT.md`.

[7] ProveIt project, *Source parallelity in all handle dimensions and a bounded inherited core*, `Topology/UnknotRecognition/synthesis/parallelity.tex`, reference revision `704600626606a9097cfd89d4e3d2bbc63c8dd1cb`.

[8] ProveIt project, *Certified boundary shellings before surface discovery*, `Topology/UnknotRecognition/synthesis/boundary_shellings.tex`, inspected 12 October 2026.

[9] ProveIt project, `Topology/UnknotRecognition/README.md`, `Topology/UnknotRecognition/reports/README.md`, and `docs/incoming/README.md`, inspected 12 October 2026. Repository: https://github.com/VladimirReshetnikov/ProveIt.

[10] Research continuation prepared for ProveIt, *A One-Scalar-Census Boundary Compiler for Compressed Unknot Hierarchies: Moment-coded first returns, exact gap measures, finite-group transport, and reusable port reservoirs*, 12 October 2026. Retrieved abstract, interface discussion and claim ledger from `ProveIt_Moment_Coded_First_Returns_2026-10-12.pdf`; not a full archive audit.